

Workout-App Implementation Master Plan

Project: Workout-App Mobile & Backend

Document Status: Sprint Planning v2.0

Prepared By: Project Management Office

Executive Summary

This document outlines the comprehensive roadmap for the development of the *Workout-App*, a premium mobile fitness application designed to help users track workouts, build routines, and monitor their health metrics. The project is broken down into 8 strategic Sprints, each consisting of 12 distinct work items giving us a total of 96 actionable tasks.

Sprint 1: Foundation Setup & User Management

Objective: Establish a rock-solid technical foundation, configure the development environment, and implement secure user authentication and onboarding flows.

1.1 Project Initialization & Monorepo Structure

Description:

Initialize the master code repository. We need a unified workspace that houses both the mobile application (React Native/Expo) and the backend API (NestJS). This structure must be clean, maintainable, and support independent scalability of both client and server.

User Story:

As a Developer, I want a well-structured monorepo so that I can efficiently manage code sharing and deployment pipelines without conflict.

Acceptance Criteria:

- Root directory contains verified `workout-mobile` and `workout-backend` folders.
- `package.json` scripts are configured to launch both environments from the root.
- Git repository is initialized with a robust `.gitignore` file.

1.2 Database Connectivity Configuration

Description:

Configure the connection between the Backend API and our PostgreSQL database. This involves setting up TypeORM (our Object-Relational Mapper) to handle database transactions safely and efficiently.

Acceptance Criteria:

- `ormconfig.ts` is created and successfully connects to the local PostgreSQL instance.
- Environment variables for database credentials are secured in `.env`.
- A test migration runs successfully, confirming write access to the DB.

1.3 User Profile Database Schema

Description:

Design and implement the core `User` database table. This is the source of truth for all account information and must be designed with security and scalability in mind from Day 1.

Acceptance Criteria:

- `users` table is created in the database.

- Essential columns included: UUID (Primary Key), Email (Unique), Password Hash, First Name, Last Name, CreatedAt, UpdatedAt.
- Constraints are applied to prevent duplicate emails.

1.4 Authentication Module & Security layer

Description:

Develop the backend logic for User Registration and Login. This includes implementing industry-standard JWT (JSON Web Token) authentication to secure our API endpoints.

Acceptance Criteria:

- API endpoint `POST /auth/register` accepts user details and creates an account.
- API endpoint `POST /auth/login` validates credentials and returns a secure access token.
- User passwords are inextricably encrypted using `bcrypt` before storage.

1.5 Mobile Application Shell

Description:

Initialize the React Native application using the Expo framework. This "shell" will serve as the container for all future mobile features.

Acceptance Criteria:

- App launches successfully on iOS Simulator.
- TypeScript integration is verified and working.
- The navigation stack (React Navigation) is installed and ready for screen configuration.

1.6 Global Design System Implementation

Description:

Translate our design language into a reusable code library. We need a central "Theme" file that defines our color palette, typography, and spacing rules to ensure visual consistency across the app.

Acceptance Criteria:

- `fitness.theme.ts` file is created containing all brand colors (Primary Blue `#1F7BFF`, etc.).
- A `ThemeProvider` wraps the application root.
- Basic text components render correctly using the defined theme fonts.

1.7 Authentication Interface (UI)

Description:

Build the user-facing screens for "Sign Up" and "Log In". These screens must be polished, user-friendly, and provide clear feedback for input errors (e.g., "Invalid Email").

Acceptance Criteria:

- Login Screen matches the high-fidelity mockups.
- Registration Screen allows users to input name, email, and password.
- Form validation is implemented (users cannot submit empty fields).

1.8 Client-Side Auth Integration

Description:

Wire up the Mobile Auth screens to the Backend API. This task allows the mobile app to actually creating accounts and logging users in.

Acceptance Criteria:

- Successful login redirects the user to the Main Dashboard.

- Login errors (wrong password, server error) are displayed in a toast message.
- The User's Session Token is stored securely on the device using `SecureStore` .

1.9 Detailed User Profile Schema

Description:

Create a secondary database table, `WorkoutProfile` , to store health-specific data (Weight, Height, Gender) separate from the core login credentials.

Acceptance Criteria:

- `workout_profiles` table created with a One-to-One relationship to the `users` table.
- Supports metric vs imperial unit preferences.

1.10 Onboarding Experience Flow

Description:

Design a seamless "Onboarding" wizard that greets new users. We need to collect their fitness goals (e.g., "Lose Weight", "Build Muscle") and physical stats to personalize their experience.

Acceptance Criteria:

- Users are automatically routed to Onboarding after their first sign-up.
- Multi-step form collects: Gender, Experience Level, Goals.
- Data is successfully saved to the `WorkoutProfile` table upon completion.

1.11 Push Notification Infrastructure

Description:

Set up the backend infrastructure to track user devices. This allows us to send push notifications later (e.g., "Time to workout!").

Acceptance Criteria:

- Database table `push_devices` created to store unique device tokens.
- API endpoint designed to register/update a device token when the app opens.

1.12 Quality Assurance & CI Pipelines

Description:

Establish code quality standards. We need automated tools (Linters, Formatters) to ensure every line of code written by the team adheres to our style guide.

Acceptance Criteria:

- Prettier and ESLint (Strict Mode) are configured for both Mobile and Backend.
- Git "Pre-commit hooks" are active (prevents committing broken code).
- Unit test framework (Jest) serves as a baseline for future testing.

Sprint 2: Exercise Library & Content Management

Objective: Build the core content engine of the application—the Exercise Database—and ensure users have access to high-quality instructional media.

2.1 Exercise Taxonomy & Data Model

Description:

Define the "Exercise" object in our system. Exercises need to be categorized intelligently (Cardio vs.

Strength) and tagged with metadata like "Muscle Group" and "Equipment Needed".

Acceptance Criteria:

- `ExerciseEntity` defined with fields: Name, Description, Type, Primary Muscle, Secondary Muscle, Equipment.
- Enumerated types supported (e.g., `Type: 'Strength' | 'Cardio'`).

2.2 Content Management API

Description:

Develop the API endpoints that allow the system to CREATE, READ, UPDATE, and DELETE (CRUD) exercises.

This serves the mobile app's library view.

Acceptance Criteria:

- `GET /exercises` returns a paginated list of exercises.
- `POST /exercises` allows Admins to add new content.
- Endpoints are secured (only authenticated users can read).

2.3 Mobile Exercise Library View

Description:

Build the "Exercise Library" screen on the mobile app. This is a browsable catalog where users can explore different movements.

Acceptance Criteria:

- Displays a vertical scrollable list of exercises.
- Each row shows an avatar (muscle icon) and the exercise name.
- Scroll performance remains smooth (60fps) even with 100+ items.

2.4 Advanced Search & Filtering

Description:

Implement robust search capabilities. Users should be able to type "Chest" or "Dumbbell" and instantly filter the library to see relevant results.

Acceptance Criteria:

- Search bar at top of Library screen filters results in real-time.
- Filter "Chips" (e.g., "Cardio", "Abs") allow quick category selection.
- "No results found" state is handled gracefully.

2.5 "Favorites" Feature - Backend

Description:

Allow users to bookmark exercises they love. We need a backend relationship to track which user has "favorited" which exercise.

Acceptance Criteria:

- Database join-table `exercise_favorites` created.
- API endpoints `POST /favorite/{id}` and `DELETE /favorite/{id}` implemented.

2.6 Exercise Detail Screen

Description:

Create the "Deep Dive" view for a single exercise. When a user taps an items in the library, they should see full instructions, tips, and media.

Acceptance Criteria:

- Screen displays large Header Video/Image.
- "How To" steps are clearly formatted.
- "Primary Muscles" are visually highlighted (tags/chips).

2.7 Video Player Component

Description:

Integrate a video player engine into the app effectively. This player will handle the instructional looping videos for exercises.

Acceptance Criteria:

- Videos auto-play on mute when the screen loads.
- Playback loops seamlessly.
- Player handles buffering states with a loading spinner.

2.8 Media Fallback System

Description:

Ensure the app looks good even if a video is missing. We need a robust fallback logic that displays a high-quality placeholder image or a default animation if the video URL fails.

Acceptance Criteria:

- If `mediaUrl` is null, a default category icon is shown.
- If a video fails to load (network error), the player is replaced by a static image functionality.
- No "Black screens" are visible to the user.

2.9 Content Seeding Script

Description:

We cannot launch with an empty library. Write an automated script to populate the database with at least 50 standard exercises (Squats, Pushups, Lunges, etc.) upon deployment.

Acceptance Criteria:

- Running `npm run seed:exercises` populates the local database.
- Exercises have correct metadata (Squat = Legs, Strength).

2.10 "Favorites" Feature - Mobile UI

Description:

Add the interactive "Heart" icon to exercise cards. Tapping it should provide instant visual feedback and sync the preference to the server.

Acceptance Criteria:

- Heart icon toggles filled/outline state instantly (Optimistic UI).
- Library can be filtered to show "Favorites Only".

2.11 Asset & Image Optimization

Description:

Optimize all static assets to ensure the app download size is small and the library loads instantly.

Acceptance Criteria:

- All exercise icons are converted to SVG or optimized WebP format.
- Large images are lazy-loaded (only loaded when scrolled into view).

2.12 Unit Testing: Exercise Module

Description:

Write automated tests to verify the reliability of the Exercise features. We need to guarantee that the search logic and data fetching work as expected.

Acceptance Criteria:

- Jest tests cover the `ExerciseService` business logic.
 - Snapshot tests created for the `ExerciseCard` component.
 - Code coverage for this module exceeds 80%.
-

Sprint 3: Routine Builder & Planning

Objective: Empower users to design their own workout programs by combining exercises into structured Routines.

3.1 Routine Architecture Design

Description:

Design the data structure for a "Routine" (a collection of exercises). A Routine needs to store the *order* of exercises and specific targets (e.g., "3 Sets of 10 Reps").

Acceptance Criteria:

- `WorkoutRoutine` database entity created (Name, Description).
- `RoutineExercise` entity created as a robust join-table containing: `OrderIndex`, `TargetSets`, `TargetReps`, `RestTime`.

3.2 Routine Management API

Description:

Build the backend endpoints for managing Routines. Users need to be able to save their custom plans, edit them later, and delete them if needed.

Acceptance Criteria:

- Endpoint `POST /routines` accepts a complex object (Routine metadata + List of Exercises).
- Endpoint `PUT /routines/{id}` allows reordering and updating targets.

3.3 Plan Builder Interface

Description:

Develop the "Create New Routine" screen. This is a form-based view where users name their routine and provide a description before adding exercises.

Acceptance Criteria:

- Input fields for "Routine Name" (Required) and "Description".
- "Save" button is disabled until valid data is entered.

3.4 Exercise Picker Modal

Description:

Create a reusable "Picker" component. When building a routine, the user clicks "Add Exercise" and sees a multi-select version of the Exercise Library.

Acceptance Criteria:

- Modal slides up from bottom.
- User can select multiple exercises at once.

- "Add (3) Exercises" button confirms selection and closes modal.

3.5 Plan Editing & Reordering

Description:

Users must be able to arrange their exercises. Implement a drag-and-drop or "Move Up/Down" interface so users can sequence their workout correctly.

Acceptance Criteria:

- List of added exercises allows reordering.
- "Swipe to Delete" allows removing an exercise from the draft routine.

3.6 Target Setting Interface

Description:

For each exercise in a routine, the user needs to define their goals. Create inputs for Sets, Reps, and Rest Intervals per exercise card.

Acceptance Criteria:

- Numeric input fields for Sets/Reps.
- Default values are provided (e.g., 3 sets, 10 reps) to save time.

3.7 Public/System Routine Favorites

Description:

Allow users to "Save" or "Copy" routines created by the System (e.g., "Beginner Full Body"). This separates User-content from System-content.

Acceptance Criteria:

- Users can "Like" a system routine, adding it to their "My Routines" list.
- Backend tracks the number of saves per routine.

3.8 "My Routines" Dashboard

Description:

Create the main view where users access their saved plans. This should be visually distinct cards showing the routine name and a summary (e.g., "5 Exercises • 45 Mins").

Acceptance Criteria:

- Grid or List view of user's routines.
- Tapping a card opens the Routine Detail view.

3.9 System Presets Implementation

Description:

Seed the database with high-quality "Starter Routines" designed by fitness professionals. Every new user should have 3-5 routines ready to go immediately.

Acceptance Criteria:

- Database populated with: "Upper Body Power", "Leg Day", "Full Body Circuit".
- System routines are read-only (users cannot edit the originals).

3.10 Routine Detail & Summary View

Description:

The "Pre-flight" screen before starting a workout. It shows the full breakdown of what the workout entails so

the user can prepare.

Acceptance Criteria:

- Displays total estimated time and muscle breakdown.
- Prominent "Start Workout" button launches the active session.

3.11 "Quick Start" Logic

Description:

Sometimes users just want to workout without a plan. Implement a "Quick Start" feature that launches an empty session immediately, skipping the Routine Builder.

Acceptance Criteria:

- "Quick Start" button on Dashboard launches a blank Workout Session.
- User can add exercises on-the-fly during the session.

3.12 Integration Testing: Routine Flow

Description:

Verify the end-to-end flow of the Plan Builder. We must ensure that complex routine objects are saved and retrieved 100% accurately.

Acceptance Criteria:

- Automated test simulates creating a routine with 3 exercises.
- Verifies that exercise order is preserved upon retrieval.

Sprint 4: Active Workout Experience (Part I)

Objective: Deliver the "Core Loop" of the application: The screen where the user actually performs the workout and logs their data in real-time.

4.1 Workout Session Data Model

Description:

Design the schema for *executed* workouts. Unlike a "Routine" (which is a plan), a "Workout" is a historical record of what actually happened (Date, Duration, Actual Reps performed).

Acceptance Criteria:

- `WorkoutEntity` captures: Start Time, End Time, Total Volume.
- `WorkoutExerciseEntity` captures actual performance data (Sets/Weights).

4.2 Session State Management (Redux)

Description:

The active workout is a complex state that must persist even if the user minimizes the app. We need a robust State Management solution (Redux) to track the timer, current exercise, and logged sets.

Acceptance Criteria:

- Redux Store configured for `ActiveSession`.
- Data persists in memory even if the user switches to Spotify and back.

4.3 Session "Hero" Interface

Description:

Build the main tracking screen. This needs to be high-energy and focused. It should show the big Timer, the

current Exercise Name, and the workout progress.

Acceptance Criteria:

- Large, readable Timer at the top.
- Progress bar showing % of workout completed.
- "Next Up" indicator showing the following exercise.

4.4 Global Workout Timer

Description:

Implement the master clock for the workout. It needs to handle "Pause" and "Resume" accurately and continue counting in the background using timestamps to prevent drift.

Acceptance Criteria:

- Timer starts immediately upon session launch.
- Tapping "Pause" stops the visual counter.
- Backgrounding the app does not stop the actual elapsed time calculation.

4.5 Exercise Navigation Controls

Description:

Give users control over their flow. They need easy buttons to move between exercises ("Next", "Previous") without losing data.

Acceptance Criteria:

- "Next Exercise" button advances the view.
- Users can swipe or tap 'Previous' to review past sets.

4.6 Active Exercise Highlighting

Description:

Visual focus is key. In the list of exercises, the *current* one should be expanded and highlighted, while others are collapsed or dimmed.

Acceptance Criteria:

- Active card has a distinct border/shadow.
- The list automatically scrolls to keep the active exercise in the center of the screen.

4.7 Set Completion Logic

Description:

The core interaction: Checking off a set. Users need a satisfying way to mark a set as "Done".

Acceptance Criteria:

- Tapping a set row toggles a Checkmark.
- Completed sets are visually dimmed.
- Marking all sets complete auto-suggests moving to the next exercise.

4.8 In-Session Video Player

Description:

Embed the instructional video directly into the active session card. Users shouldn't have to leave their logging screen to check their form.

Acceptance Criteria:

- Small video player embedded in the active exercise card.

- Tap-to-fullscreen option available.

4.9 Session Safety Checks (Cancel/Quit)

Description:

Prevent accidental data loss. If a user tries to close the workout or swipe back, intercept the action and ask for confirmation.

Acceptance Criteria:

- "Are you sure?" alert appears on back button press.
- Options provided: "Resume Workout", "Finish & Save", "Discard".

4.10 Workout Saving Implementation

Description:

The "Finish Line". When a user completes a workout, package up all the complex data (hundreds of data points) and transmit it securely to the Backend API.

Acceptance Criteria:

- `POST /workout/history` saves the full session payload.
- Transaction ensures all-or-nothing saving (no partial workouts).

4.11 Workout Summary Screen

Description:

After finishing, show a "Recap" screen. Display key stats like "Total Time", "Total Weight Lifted", and "Calories Burned" to give the user a sense of accomplishment.

Acceptance Criteria:

- Summary screen appears immediately after finishing.
- User can add specific notes (e.g., "Felt strong today") before final save.

4.12 Timer Reliability Testing

Description:

Rigorous testing of the timer mechanics. We must ensure that if a user works out for 45 minutes, the app records exactly 45 minutes, regardless of phone handling.

Acceptance Criteria:

- Timer accuracy verified against external stopwatch.
- Verified behavior when receiving a phone call (Interruption handling).

Sprint 5: Active Workout Experience (Part II)

Objective: Refine the workout experience with "Delight" features, offline capabilities, and advanced logging options.

5.1 Offline Mode Support

Description:

Users in gym basements often have poor signal. The app must allow starting, logging, and finishing a workout completely offline.

Acceptance Criteria:

- App detects "No Internet" state.

- Completed workouts are saved to local device storage (`AsyncStorage`) instead of failing.

5.2 Background Sync Mechanism

Description:

Build a robust sync engine. When the internet connection is restored, the app should automatically detect pending offline workouts and upload them to the cloud.

Acceptance Criteria:

- Sync triggers automatically on connection restore.
- User is notified "Workout Synced" via a toast message.

5.3 Audio Feedback System

Description:

Add audio cues to the workout. Users should hear a "Beep" or "Whistle" when a timer ends, so they don't have to stare at their screen during a plank.

Acceptance Criteria:

- Sound effects play for: Timer Start, Timer End, Workout Complete.
- "Mute" option available in settings.

5.4 Haptic Feedback Integration

Description:

Utilize the phone's vibration motor to provide tactile feedback. This makes the app feel responsive and physical.

Acceptance Criteria:

- Light vibration on button taps.
- Heavy vibration when a timer completes.

5.5 Calorie Burn Algorithm

Description:

Implement a smart estimation logic for Calories Burned. This should take into account the workout duration, intensity (Rest times), and the user's weight.

Acceptance Criteria:

- Algorithm differentiates between "Strength" (lower burn) and "Cardio" (higher burn).
- Estimated value shown on Summary screen.

5.6 Exercise Note Taking

Description:

Detailed logging. Allow users to write specific notes on an exercise level (e.g., "Raise seat height to 4 next time").

Acceptance Criteria:

- Text input field available on every exercise card.
- Notes are saved and resurfaced the next time the user performs that exercise.

5.7 Dynamic Session Editing

Description:

Flexibility is key. Allow users to deviate from the plan *during* the workout. They should be able to add a new

exercise or remove one if the machine is taken.

Acceptance Criteria:

- "Add Exercise" button available inside the active session.
- Swipe-to-delete available for pending exercises.

5.8 Rest Timer Automation

Description:

When a user checks off a set, automatically trigger a "Rest Timer". This keeps the user on pace.

Acceptance Criteria:

- Rest timer overlay appears immediately after completing a set.
- Users can add/subtract 30s to the timer easily.
- "Skip Rest" button available.

5.9 "Keep Awake" Screen Lock

Description:

Prevent the screen from turning off during a workout. It is frustrating to have to unlock the phone with sweaty hands between sets.

Acceptance Criteria:

- App requests "Wake Lock" when a session is active.
- Screen dims but does not lock (to save battery) if supported.

5.10 Workout History List

Description:

Build the "History" tab where users can scroll back through their training logs.

Acceptance Criteria:

- Paginated list of all past workouts (Newest first).
- Each row summarizes the session (Date, Name, Volume).

5.11 Celebration Interactions

Description:

Add a moment of joy at the finish line. When a workout is saved, use animation (e.g., Confetti) to congratulate the user.

Acceptance Criteria:

- Lottie animation triggers on the Summary screen.
- Success message is enthusiastic and encouraging.

5.12 Code Refactoring & Optimization

Description:

Mid-project review. Take this sprint to clean up the `WorkoutSession` code, which has likely grown complex. Break it down into smaller, performant components.

Acceptance Criteria:

- `WorkoutSession.tsx` file size reduced by extracting components.
 - No "prop drilling" issues; effectively using Redux selectors.
-

Sprint 6: Analytics & Progress Dashboard

Objective: Visualize the user's hard work. Provide data-driven insights and a compelling dashboard that encourages retention.

6.1 Dashboard Statistics Backend

Description:

Create a high-performance aggregation endpoint. The dashboard needs to load fast and show "Total Workouts", "Current Streak", and "Active Minutes".

Acceptance Criteria:

- `GET /dashboard/stats` returns pre-calculated metrics.
- Database queries are optimized to avoid scanning the entire history table every time.

6.2 Intelligent Streak Logic

Description:

Gamify the experience. Calculate the user's "Streak" (consecutive days/weeks active). Logic must be forgiving of timezones.

Acceptance Criteria:

- Streak counter increments after a workout.
- Logic handles "Rest Days" if configured (optional).

6.3 Body Metrics Tracking

Description:

Allow users to track their physical changes, not just their gym performance. Create a system for logging Weight, Body Fat %, etc.

Acceptance Criteria:

- `ProgressEntry` entity created.
- Users can log a new weight entry with a date.

6.4 Main Dashboard Interface

Description:

Assemble the "Home" screen. This is the command center, showing the user's current status, streak, and quick access to their next workout.

Acceptance Criteria:

- "Weekly Momentum" card shows a visual representation of the week's activity.
- "Focus Spotlight" shows which muscle groups are being trained most.

6.5 Data Visualization (Charts)

Description:

Implement visual charts. A line graph showing "Body Weight over Time" or accurate bar charts for "Workouts per Week".

Acceptance Criteria:

- SVG Charts implemented gracefully.
- Charts handle empty data states without crashing.
- Users can toggle timeframes (1 Month, 3 Months, 1 Year).

6.6 Measurement Logging UI

Description:

A dedicated screen for inputting body metrics. Needs to be simple and quick.

Acceptance Criteria:

- Input form for Weight/Body Fat.
- Date picker defaults to "Today".

6.7 Progress API Endpoints

Description:

Endpoints to fetch the history of body metrics for the graphing engine.

Acceptance Criteria:

- `GET /progress/weight` returns time-series data.
- Data is sorted chronologically.

6.8 "Recent Activity" Feed

Description:

A mini-feed on the dashboard showing the last 3 activities. This serves as a quick shortcut to view details or resume a draft.

Acceptance Criteria:

- Displays the 3 most recent entries.
- Tapping opens the Workout Detail view.

6.9 Calendar History View

Description:

A visual calendar widget where days with workouts are marked with a dot or color. This gives a great high-level view of consistency.

Acceptance Criteria:

- Monthly calendar view.
- "Active Days" are highlighted.
- Tapping a day shows the workout performed on that date.

6.10 Personal Records (PR) Engine

Description:

Automatically calculate "Personal Bests". When a user finishes a workout, check if they lifted a heavier weight than ever before.

Acceptance Criteria:

- System flags "New 1RM" (One Rep Max) or "Best Volume".
- PR badge appears on the exercise history.

6.11 Achievements & Badges

Description:

Visual rewards system. Create static badges for milestones like "First Workout", "7 Day Streak", "10,000 lbs Lifted".

Acceptance Criteria:

- "Achievements" section in profile.

- Badges unlock automatically based on stats.

6.12 Data Verification QA

Description:

Ensure the numbers are real. If the dashboard says "15 Workouts", there must be exactly 15 records.

Acceptance Criteria:

- Manual audit of database vs frontend stats.
 - Verify charts render correct scaling.
-

Sprint 7: Sleep Tracker & Health Integration

Objective: Expand the app's scope to holistic health by integrating Sleep Tracking, specifically leveraging iOS HealthKit.

7.1 HealthKit Configuration & Permissions

Description:

Prepare the iOS app for Health connections. This involves configuring the `Info.plist` with privacy strings explaining *why* we need access to health data.

Acceptance Criteria:

- App successfully requests "Read/Write" permissions for Sleep Analysis.
- Privacy prompt explains usage clearly to the user.

7.2 Sleep Service Module

Description:

Develop the internal service layer that communicates with Apple HealthKit. It needs to abstract the complex Apple API into simple methods for our app.

Acceptance Criteria:

- `getSleepData(startDate, endDate)` returns normalized sleep samples.
- `saveSleepData(entry)` writes data back to Apple Health.

7.3 Permission Onboarding Flow

Description:

We cannot just pop a permission dialog. Build a "Setup" card on the Dashboard that explains the benefits of connecting HealthKit before asking.

Acceptance Criteria:

- "Connect Sleep Tracker" card appears for new users.
- Grants access -> Card transforms into Data View.
- Denies access -> Graceful fallback UI.

7.4 Local Sleep Caching

Description:

HealthKit querying can be slow. Implement a local caching strategy so the sleep data loads instantly when the app opens, then updates in the background.

Acceptance Criteria:

- Last known sleep data is stored in `AsyncStorage`.

- Cached data is displayed immediately on launch.

7.5 Sleep Tracker Dashboard Card

Description:

Visual component for the Home screen. Should show "Hours Slept" last night and a small trend indicator.

Acceptance Criteria:

- Shows "8h 12m" style duration.
- "Sleep Score" or quality metric if available.

7.6 Manual Sleep Entry

Description:

Not everyone wears a watch to bed. Allow users to manually input their start and wake times.

Acceptance Criteria:

- Time pickers for "Bed Date/Time" and "Wake Date/Time".
- Saves data to HealthKit as a "Sleep Analysis" sample.

7.7 Sleep Trends Charting

Description:

Visual analysis of sleep. A bar chart showing the last 7 days of sleep duration.

Acceptance Criteria:

- Bar chart with "Goal Line" (e.g., 8 hours).
- Visual distinction between "Deep Sleep" vs "Light Sleep" if data effectively available.

7.8 Cloud Backups (Sleep)

Description:

(Optional) Sync key sleep metrics to our backend database for long-term trends and cross-device analysis.

Acceptance Criteria:

- Sleep summary data is included in the user's daily sync.
- Data is anonymized where appropriate.

7.9 Android Platform Handling

Description:

Since HealthKit is iOS only, we must ensure the Android version of the app doesn't crash or show broken features.

Acceptance Criteria:

- Platform checks hide HealthKit features on Android.
- Android users see a "Coming Soon" or alternative manual tracker (if scoped).

7.10 Foreground Data Refresh

Description:

If a user wakes up and checks Apple Health, then opens our app, our data must update immediately.

Acceptance Criteria:

- App listens for `AppState` changes (Background -> Active).
- Triggers a HealthKit re-fetch on activation.

7.11 Sleep Insights Engine

Description:

Provide value, not just numbers. extensive logic to analyze the sleep data and give tips (e.g., "Your sleep average is down 10% this week").

Acceptance Criteria:

- Logic calculates 7-day rolling average.
- Displays insight text on the Sleep Detail view.

7.12 Device Testing

Description:

Verification on real hardware. The iOS Simulator does not perfectly emulate HealthKit data stores.

Acceptance Criteria:

- Verified on a physical iPhone.
- Verified permissions persist after app restart.

Sprint 8: Polish, Configuration & Reminders

Objective: The Final Polish. Focus on user retention loops (Reminders), Settings, Branding, and preparing a production-ready build.

8.1 Workout Reminder Architecture

Description:

Build the backend structure for user preferences regarding reminders. Users should be able to say "Remind me on Mon/Wed/Fri at 8 AM".

Acceptance Criteria:

- `WorkoutReminder` entity stores: Days of Weeks, Time, Local Timezone.

8.2 Notification Scheduling System

Description:

Implement the Cron/Scheduler jobs that actually dispatch the notifications. This needs to handle thousands of users in different timezones.

Acceptance Criteria:

- Job runs hourly.
- Identifies users who requested a reminder in the current hour window.
- Dispatches Push Notification via Firebase/Expo.

8.3 Reminder Settings UI

Description:

User interface for configuring the schedule. "Days of Week" selector and "Time Picker".

Acceptance Criteria:

- Toggle "Enable Reminders".
- Multi-select for Mon-Sun.
- Settings save instantly to backend.

8.4 Comprehensive Settings Panel

Description:

Build the full "Settings" page. This is the utility drawer for the app.

Acceptance Criteria:

- "Edit Profile" (Name, Avatar).
- "Unit Preferences" (Metric vs Imperial).
- "Sign Out" (Clear tokens and reset state).
- "Delete Account" (GDPR requirement).

8.5 Branding & App Assets

Description:

Replace all default "Expo" icons with our premium "Workout-App" branding.

Acceptance Criteria:

- App Icon (iOS/Android) generated in all required resolutions.
- Splash Screen configured with brand logo and background color.

8.6 Deep Linking Configuration

Description:

Enable the app to open from URL links. If a user gets a "Workout Reminder" notification, tapping it should take them directly to the `PlanBuilder` or `Session` screen.

Acceptance Criteria:

- URL Scheme `workout://` configured.
- Route `workout://session/quick` launches Quick Start.

8.7 Performance Profiling & Optimization

Description:

A final sweep for performance. Identify dropped frames or slow lists and fix them using `React.memo` and optimized list keys.

Acceptance Criteria:

- `ExerciseLibrary` scrolls at 60fps on iPhone 12 or equivalent.
- App launch time (Cold Start) is under 2 seconds.

8.8 Legal & Compliance Views

Description:

Add the boring but necessary screens. "Privacy Policy", "Terms of Service", and Open Source Licenses.

Acceptance Criteria:

- "About" section in Settings.
- WebViews or Text Views displaying the legal copy.

8.9 Release Pipeline (Fastlane)

Description:

Automate the build process. We don't want to manually archive builds. Set up Fastlane to build and upload to TestFlight with one command.

Acceptance Criteria:

- `fastlane ios beta` command builds the app and uploads to App Store Connect.
- Version numbers auto-increment.

8.10 Production Environment Hardening

Description:

Prepare the backend for the real world. Ensure debug logs are disabled, SSL is enforced, and production database credentials are secure.

Acceptance Criteria:

- `production.env` created.
- Dependency audit run (`npm audit`) to fix vulnerabilities.

8.11 Pre-Release Bug Sweep

Description:

An all-hands-on-deck QA pass. Click every button, try every edge case (offline, airplane mode, kill app). Fix the visual glitches.

Acceptance Criteria:

- Zero "Critical" or "High" priority bugs remaining.
- Keyboard avoidance works on all input forms.

8.12 Initial User Guide & Documentation

Description:

Write the manual. A "User Guide" markdown file or Help Center article explaining the core features.

Acceptance Criteria:

- `USER_GUIDE.md` created.
- Explains: How to create a routine, how to track a workout, how to sync HealthKit.